

HIGH LEVEL DESIGN (HLD)

1. System Overview

Problem Definition:

Traditional customer support systems struggle to provide accurate and context-aware answers from large knowledge bases such as PDFs, FAQs, and policy documents. They often return generic or irrelevant responses and lack the ability to handle complex queries effectively. These limitations reduce customer satisfaction and increase dependency on human agents, leading to higher operational costs.

Proposed Solution:

This project designs a Retrieval-Augmented Generation (RAG)-based Customer Support Assistant that:

- Extracts information from a PDF knowledge base.
- Retrieves relevant context using embeddings.
- Generates accurate answers using a Large Language Model (LLM).
- Uses a graph-based workflow (LangGraph) for decision-making.
- Supports Human-in-the-Loop (HITL) escalation for complex or low-confidence queries.

The system enhances response accuracy by grounding LLM outputs in retrieved knowledge instead of relying solely on pretrained data.

Scope of the System:

- Supports single PDF input (extendable to multiple documents).
 - Provides a CLI/Web-based interface for interaction.
 - Handles real-time user queries.
 - Performs context-aware response generation.
 - Includes escalation mechanism for low-confidence responses.
-

2. Architecture Diagram

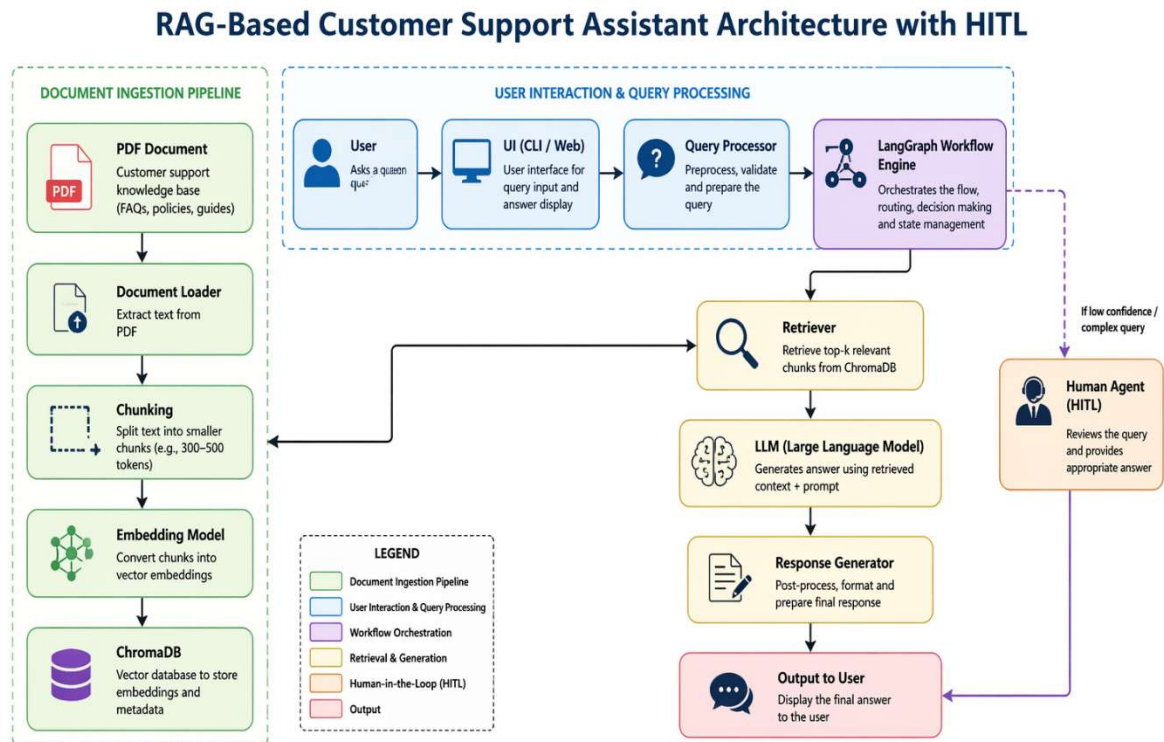


Figure: Architecture of RAG-Based Customer Support Assistant with LangGraph and HITL

Architecture Explanation:

The system consists of two main pipelines: a document processing pipeline and a query processing pipeline. In the document pipeline, the input PDF is processed through loading, chunking, and embedding stages before being stored in the ChromaDB vector database. In the query pipeline, user queries are processed through the LangGraph workflow, which orchestrates retrieval and response generation. The Retriever fetches relevant document chunks, which are passed to the LLM for generating context-aware responses. If the system detects low confidence or insufficient context, the query is escalated to the Human-in-the-Loop (HITL) module for manual intervention. This architecture ensures accurate, efficient, and reliable customer support.

3. Component Description

1. Document Loader

Loads and extracts text from PDF documents for further processing.

2. Chunking Strategy

Splits the document into smaller chunks (typically 300–500 tokens) to improve retrieval accuracy and maintain contextual relevance.

3. Embedding Model

Converts text chunks into numerical vector representations that capture semantic meaning, enabling similarity-based search.

4. Vector Store (ChromaDB)

Stores embeddings efficiently and supports fast similarity-based retrieval of relevant document chunks.

5. Retriever

Fetches top-k relevant chunks from the vector database based on the semantic similarity of the user query.

6. LLM (Large Language Model)

Generates context-aware and human-like responses using the retrieved information and user query.

7. Graph Workflow Engine (LangGraph)

Controls the system workflow, including execution flow, state management, and decision-making logic.

8. Routing Layer

Evaluates the system's confidence and determines whether to generate a response automatically or escalate the query.

9. HITL Module (Human-in-the-Loop)

Handles escalation by forwarding complex or uncertain queries to a human agent and integrating their response into the system output.

4. Data Flow

Document Processing Flow:

1. The PDF document is uploaded into the system.
2. Text is extracted using the Document Loader.
3. The extracted text is divided into smaller chunks.
4. Each chunk is converted into embeddings using the embedding model.
5. The embeddings are stored in the ChromaDB vector database.

Query Lifecycle

1. The user submits a query through the interface.

2. The query is processed and converted into an embedding.
3. The Retriever identifies the most relevant document chunks from ChromaDB.
4. Retrieved context is passed to the LLM.
5. The LLM generates a context-aware response.
6. The system evaluates response confidence and relevance.
7. If confidence is low or query is complex, it is escalated to the HITL module.
8. The final response (automated or human-assisted) is returned to the user.

This ensures that responses are both contextually relevant and reliable.

5. Technology Choices

ChromaDB

- Lightweight and efficient vector database.
- Supports fast similarity search.
- Easy integration with Python-based frameworks.
- Suitable for local and scalable deployment.

LangGraph

- Enables graph-based workflow orchestration.
- Supports conditional routing and state transitions.
- Ideal for implementing decision-driven AI systems.

LLM Choice (e.g., GPT / Llama)

- Strong natural language understanding capabilities.
- Generates accurate, context-aware, and human-like responses.
- Selected for its ability to process retrieved context effectively.

Additional Tools

- Python → Core development language
 - LangChain → Simplifies RAG pipeline implementation
 - Streamlit (optional) → User interface development
-

6. Scalability Considerations

Handling Large Documents

- Efficient chunking strategies to maintain context.
- Optimized indexing for faster retrieval.
- Incremental processing for large datasets.

Increasing Query Load

- Parallel query handling.
- Caching frequently asked queries.
- Scalable backend architecture.

Latency Optimization

- Limiting retrieval to top-k relevant chunks.
- Reducing prompt size for faster LLM response.
- Using efficient embedding models.

System Scalability

The system is designed to scale horizontally with increasing data volume and user demand, ensuring consistent performance.